Edge Intelligence Assignment 3

Dhayanidhi R S

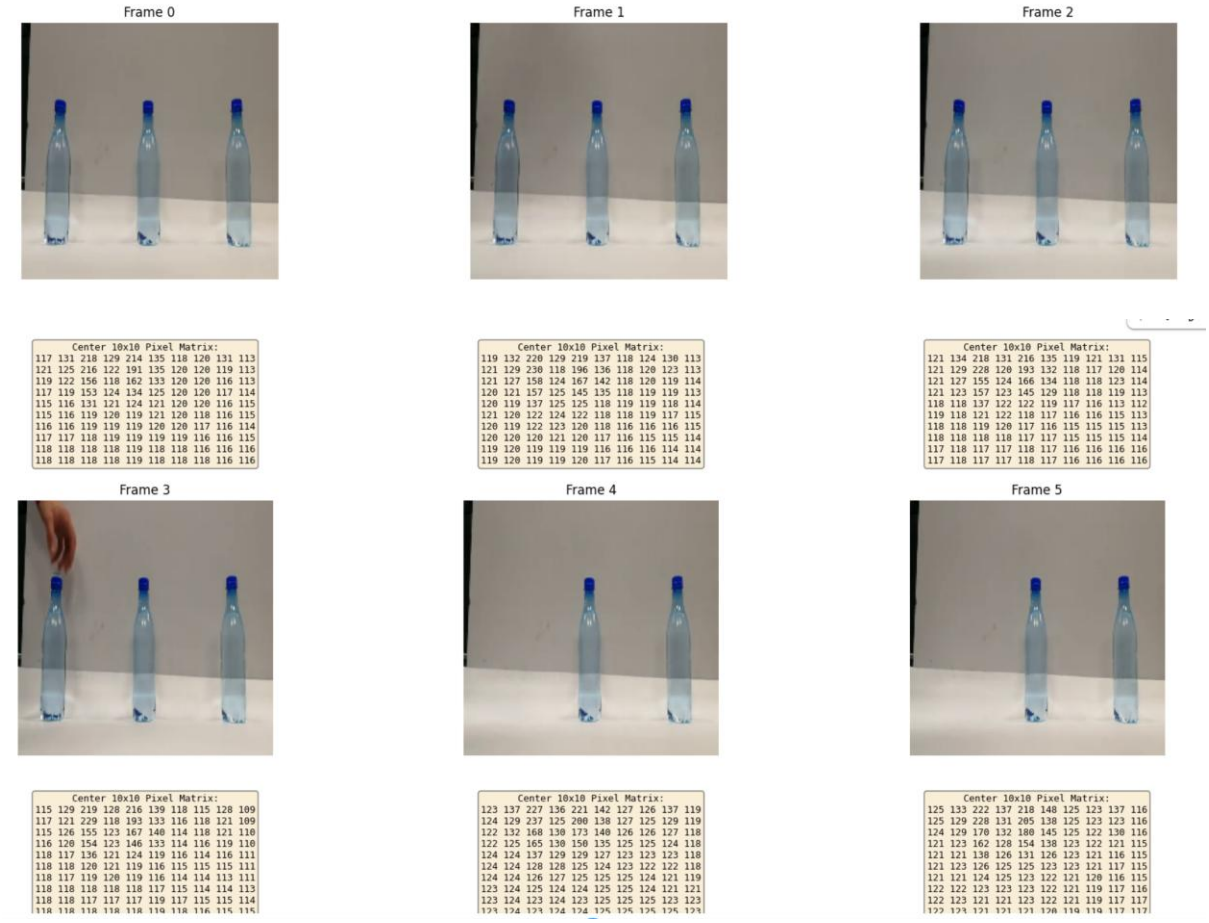
25MML0025

Lab 7 Task:

```python
import cv2
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input, decode_predictions
import matplotlib.pyplot as plt

# Load pretrained MobileNetV2
model = MobileNetV2(weights='imagenet')

def extract_frames(video_path, max_frames=20):
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        print("Error: Cannot open video")
        return np.array([])

    frames = []
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    step = max(1, total_frames // max_frames) if total_frames > 0 else 1
    count = 0

    while True:
        ret, frame = cap.read()
        if not ret: break
        if count % step == 0:
            frame = cv2.resize(frame, (224, 224))
            frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
            frames.append(frame)
        count += 1
        if len(frames) >= max_frames: break

    cap.release()
    return np.array(frames)
```

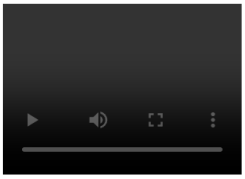
```python
def classify_video(video_path):
    frames = extract_frames(video_path)
    if len(frames) == 0: return

    cols = 3 # Reduced columns to make space for the text matrix
    rows = (len(frames) + cols - 1) // cols

    # Increase height to accommodate the text blocks
    fig, axes = plt.subplots(rows, cols, figsize=(18, 6 * rows))
    axes = axes.flatten()

    for i in range(len(axes)):
        if i < len(frames):
            frame = frames[i]

            # 1. Display Frame
            axes[i].imshow(frame)
            axes[i].set_title(f"Frame {i}")
            axes[i].axis('off')

            # 2. Extract Center 10x10 Matrix (Red Channel)
            # We use the Red channel (0) to represent the intensity
            start, end = 107, 117
            pixel_matrix = frame[start:end, start:end, 0]

            # 3. Format matrix as string and print below the image
            matrix_str = "\n".join([" ".join([f"{val:3}" for val in row]) for row in pixel_matrix])
            axes[i].text(0.5, -0.15, f"Center 10x10 Pixel Matrix:\n{matrix_str}",
                         transform=axes[i].transAxes, fontsize=9,
                         family='monospace', ha='center', va='top',
                         bbox=dict(boxstyle="round", facecolor='wheat', alpha=0.5))
        else:
            axes[i].axis('off')

    plt.tight_layout()
    plt.show()
```

```
    # --- Classification ---
    processed_frames = preprocess_input(frames.copy().astype(float))
    predictions = model.predict(processed_frames)
    avg_pred = np.mean(predictions, axis=0)
    top_preds = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)[0]

    print("\n" + "="*30)
    print("      FINAL PREDICTIONS")
    print("="*30)
    for _, label, prob in top_preds:
        print(f"{label:15}: {prob*100:.2f}%")

    print(f"\nFinal Prediction: {top_preds[0][1]}")

# Run the function
video_path = "/content/bottle-detection.mp4"
classify_video(video_path)
```




```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from glob import glob
import IPython.display as ipd
from tqdm.notebook import tqdm
```

```
ipd.Video("/content/bottle-detection.mp4", width=200)
```



```
# Load in video capture
import cv2
cap = cv2.VideoCapture('/content/bottle-detection.mp4')
```

```
# Total number of frames in video
cap.get(cv2.CAP_PROP_FRAME_COUNT)
```

```
1189.0
```

```python
# Video height and width
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
print(f'Height {height}, Width {width}')
```

```
Height 360.0, Width 640.0
```

```python
# Get frames per second
fps = cap.get(cv2.CAP_PROP_FPS)
print(f'FPS : {fps:0.2f}')
```

```
FPS : 29.83
```

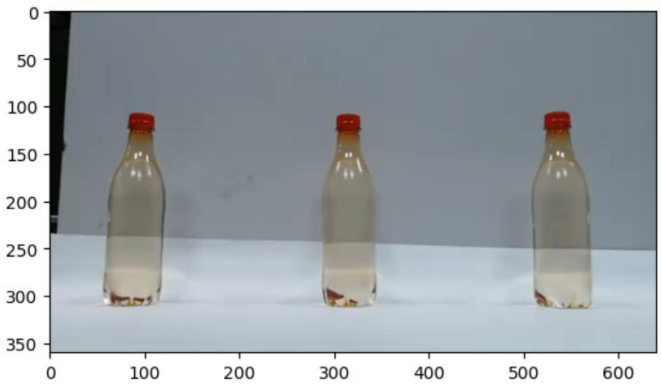
```python
cap.release()
```

```python
cap = cv2.VideoCapture('/content/bottle-detection.mp4')
ret, img = cap.read()                          #will return TRUE everytime we call this function till we reach the end of the video
print(f'Returned {ret} and img of shape {img.shape}')
```

```
Returned True and img of shape (360, 640, 3)
```

```python
plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x7c86b7806000>
```

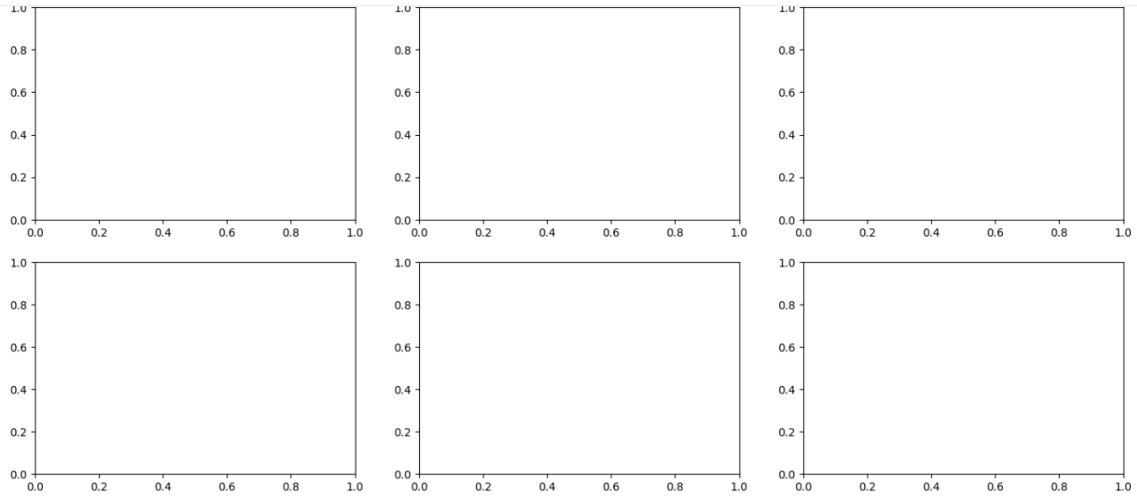



```python
## Helper function for plotting opencv images in notebook
def display_cv2_img(img, figsize=(10, 10)):
    img_ = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    fig, ax = plt.subplots(figsize=figsize)
    ax.imshow(img_)
    ax.axis("off")
```

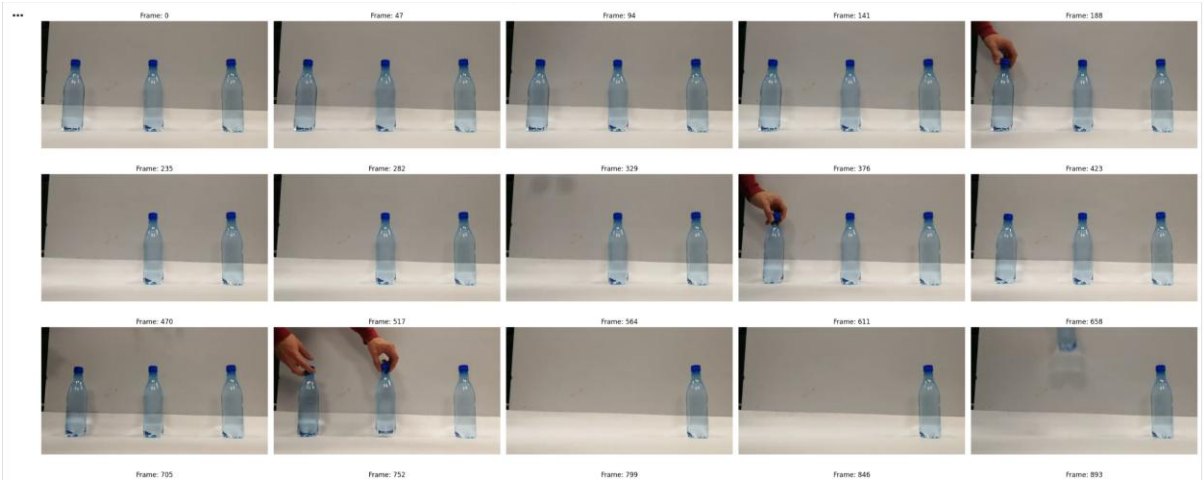
```python
display_cv2_img(img)
```

```python
cap.release()
```

```
fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()
plt.show()

# Without flatten → 2D grid (row, column)
# With flatten → 1D list (single index)
```



```
import cv2
import matplotlib.pyplot as plt

fig, axs = plt.subplots(5, 5, figsize=(30, 20))
axs = axs.flatten()

cap = cv2.VideoCapture("/content/bottle-detection.mp4")
n_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))

# Calculate step to get approximately 25 evenly spaced frames
num_frames_to_show = len(axs) # 25 frames for a 5x5 grid
step = max(1, n_frames // num_frames_to_show)

img_idx = 0
for frame_count in range(n_frames):
    ret, img = cap.read()
    if not ret: # Use 'not ret' for cleaner check
        break

    # Display a frame if it's at the calculated step and we haven't filled all subplots
    if frame_count % step == 0 and img_idx < num_frames_to_show:
        axs[img_idx].imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        axs[img_idx].set_title(f'Frame: {frame_count}')
        axs[img_idx].axis('off')
        img_idx += 1

# Turn off any unused subplots if the video has too few frames
for i in range(img_idx, num_frames_to_show):
    axs[i].axis('off')

plt.tight_layout()
plt.show()
cap.release()
```



Lab 8 Task:

```
import cv2

cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)

print("Camera opened: ", cap.isOpened())
```

```
Camera opened:  False
```

Step 1: Import Libraries

We start by importing the necessary libraries: `cv2` for OpenCV operations and `matplotlib.pyplot` for displaying images.

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
```

Step 2: Load the Video

We'll load the sample video located at `/content/sample_video.mp4` using `cv2.VideoCapture`. We then print some basic information about the video, such as total frames, frames per second (fps), width, and height.

```
cap = cv2.VideoCapture("/content/sample_video.mp4")

total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
print("Total number of frames: ", total_frames)

fps = cap.get(cv2.CAP_PROP_FPS)
print("Frames per second: ", fps)

width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)

print("width: ", width)
print("height: ", height)
```

```
Total number of frames:  600
Frames per second:  60.0
width:  1920.0
height:  1080.0
```

Step 3: Extract and Display the First Frame

To extract the first frame, we set the video position to 0 and then read the frame. OpenCV reads images in BGR format, so we convert it to RGB for correct display with `matplotlib`.

```
# Set video to the first frame
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

# Read the first frame
ret, frame = cap.read()

if ret:
    # Convert the frame from BGR to RGB for displaying with matplotlib
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Display the frame
    plt.imshow(frame_rgb)
    plt.title('First Frame of the Video (RGB)')
    plt.axis('off') # Hide axes ticks and labels
    plt.show()
else:
    print("Could not read the first frame.")
```

Step 4: Convert the First Frame to Grayscale and RGB, and Print Pixel Values

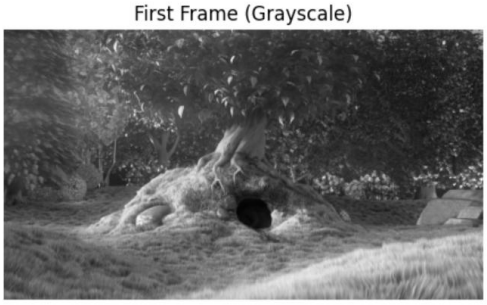
Now, we will convert the first frame to grayscale and also keep the original RGB version. We'll then print a small section of their pixel values to inspect them.

```
if ret:
    # Convert to Grayscale
    frame_gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Print a small section of RGB frame pixel values (first 5x5 pixels of each channel)
    print("\n--- First Frame (RGB) Pixel Values (top-left 5x5) ---")
    print("Red channel:\n", frame_rgb[:5, :5, 0])
    print("Green channel:\n", frame_rgb[:5, :5, 1])
    print("Blue channel:\n", frame_rgb[:5, :5, 2])

    # Print a small section of Grayscale frame pixel values (first 5x5 pixels)
    print("\n--- First Frame (Grayscale) Pixel Values (top-left 5x5) ---")
    print(frame_gray[:5, :5])

    # Display both images for visual comparison
    plt.figure(figsize=(12, 6))
    plt.subplot(1, 2, 1)
    plt.imshow(frame_rgb)
    plt.title('First Frame (RGB)')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(frame_gray, cmap='gray')
    plt.title('First Frame (Grayscale)')
    plt.axis('off')
    plt.show()
else:
    print("Frame was not read successfully, cannot convert or print pixel values.")
```

```
--- First Frame (RGB) Pixel Values (top-left 5x5) ---
Red channel:
 [[48 48 47 47 47]
 [48 48 47 47 47]
 [45 45 44 44 44]
 [45 45 44 44 44]
 [72 77 75 68 62]]
Green channel:
 [[54 54 53 53 53]
 [54 54 53 53 53]
 [56 56 55 55 55]
 [56 56 55 55 55]
 [87 92 90 83 77]]
Blue channel:
 [[41 41 40 40 40]
 [41 41 40 40 40]
 [41 41 40 40 40]
 [41 41 40 40 40]
 [71 76 74 67 61]]

--- First Frame (Grayscale) Pixel Values (top-left 5x5) ---
[[51 51 50 50 50]
 [51 51 50 50 50]
 [51 51 50 50 50]
 [51 51 50 50 50]
 [81 86 84 77 71]]
```

First Frame (RGB)

First Frame (Grayscale)



Step 5: Release the Video Capture Object

It's important to release the video capture object when you're done with it to free up resources.

```
cap.release()
print("Video capture object released.")
```

```
Video capture object released.
```

Step 6: Draw Shapes on a Frame

We will re-initialize the video capture object and attempt to read a frame. If a frame cannot be read (due to the previously identified issue), a dummy black frame will be created. We will then draw a rectangle, a circle, and a polygon on this frame.

```
# Re-initialize video capture (as it was released earlier)
cap = cv2.VideoCapture('/content/sample_video.mp4')

if not cap.isOpened():
    print("Error: Could not re-open video. Creating a dummy frame for drawing.")
    # Create a dummy black frame if video cannot be opened
    # Dimensions from previous successful read: width=1920, height=1080
    frame_to_draw = np.zeros((1080, 1920, 3), dtype=np.uint8)
    # Make it a dark grey so shapes are visible
    frame_to_draw.fill(50)
else:
    ret, frame_to_draw = cap.read()
    if not ret:
        print("Warning: Video opened but could not read frame. Creating a dummy frame for drawing.")
        # Create a dummy black frame if frame cannot be read
        frame_to_draw = np.zeros((1080, 1920, 3), dtype=np.uint8)
        frame_to_draw.fill(50)
    else:
        print("Successfully read a frame from the video.")
```

```python
# Convert to RGB for Matplotlib display
frame_to_draw_rgb = cv2.cvtColor(frame_to_draw, cv2.COLOR_BGR2RGB)
frame_with_shapes = frame_to_draw_rgb.copy()

# --- Drawing text ---
font = cv2.FONT_HERSHEY_SIMPLEX
font_scale = 2
font_thickness = 4
text = "Sample Detected Objects"
text_color = (255, 255, 255) # White color for text
text_position = (50, 80) # Top-left corner

frame_with_shapes = cv2.putText(frame_with_shapes, text, text_position, font, font_scale, text_color, font_thickness, cv2.LINE_AA)

# --- Drawing shapes ---

# 1. Draw a rectangle
# Coordinates: top-left (x1, y1), bottom-right (x2, y2)
# Color: Green (0, 255, 0) in BGR for OpenCV, but we are drawing on RGB after conversion
# Thickness: 2 pixels
start_point_rect = (200, 100)
end_point_rect = (600, 500)
color_rect = (0, 255, 0) # Green in RGB
thickness_rect = 5
frame_with_shapes = cv2.rectangle(frame_with_shapes, start_point_rect, end_point_rect, color_rect, thickness_rect)

# 2. Draw a circle
# Center: (x, y), Radius: r
# Color: Blue (0, 0, 255) in RGB
center_circle = (1000, 300)
radius_circle = 200
color_circle = (0, 0, 255) # Blue in RGB
thickness_circle = 5
frame_with_shapes = cv2.circle(frame_with_shapes, center_circle, radius_circle, color_circle, thickness_circle)
```

```python
# 3. Draw a polygon (representing a custom bounding box)
# Points are (x, y) coordinates
pts = np.array([[80, 700], [200, 900], [400, 850], [300, 650]], np.int32)
pts = pts.reshape((-1, 1, 2))
color_polygon = (255, 0, 255) # Magenta in RGB
thickness_polygon = 5
# isClosed specifies whether the polygon is closed or not (draws last line to first point)
frame_with_shapes = cv2.polylines(frame_with_shapes, [pts], isClosed=True, color=color_polygon, thickness=thickness_polygon)

# Display the frame with shapes
plt.figure(figsize=(15, 10))
plt.imshow(frame_with_shapes)
plt.title('Frame with Drawn Shapes and Text')
plt.axis('off')
plt.show()
```

Frame with Drawn Shapes and Text

Step 7: Release the Video Capture Object

It's important to release the video capture object when you're done with it to free up resources.

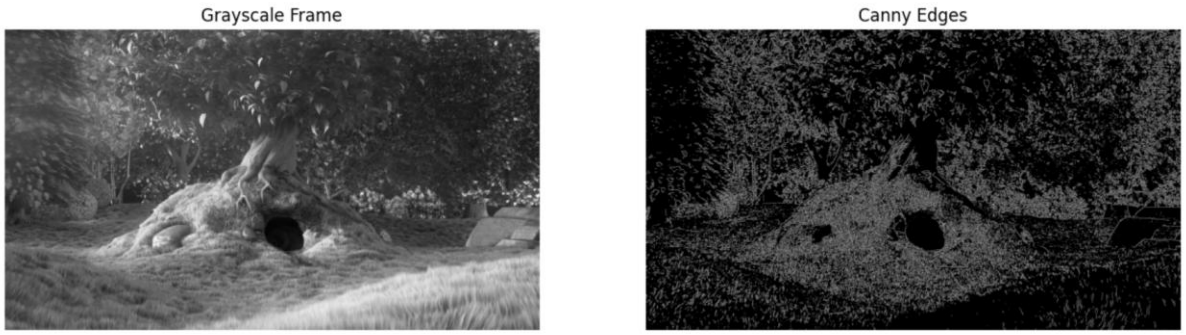
```
# Release the video capture object
cap.release()
print("Video capture object released.")
```

```
Video capture object released.
```

Step 8: Implement Canny Edge Detection

Now we will apply the Canny edge detection algorithm to our frame. Canny is a multi-stage algorithm that detects a wide range of edges in images. It's important to convert the image to grayscale before applying Canny for optimal results.

```
# Convert the frame to grayscale (Canny typically works on grayscale images)
# Using frame_to_draw_rgb which is the RGB version of the frame that was read.
frame_gray_for_canny = cv2.cvtColor(frame_to_draw_rgb, cv2.COLOR_RGB2GRAY)

# Apply Canny edge detection
# The arguments are the image, and two threshold values (threshold1 and threshold2).
# Any edges with intensity gradient more than threshold2 are sure to be edges,
# and those below threshold1 are sure to be non-edges.
# Edges with intensity gradient between threshold1 and threshold2 are classified
# as edges or non-edges based on their connectivity.
threshold1 = 100
threshold2 = 100
edges = cv2.Canny(frame_gray_for_canny, threshold1, threshold2)

# Display the original grayscale frame and the Canny edges
plt.figure(figsize=(15, 7))

plt.subplot(1, 2, 1)
plt.imshow(frame_gray_for_canny, cmap='gray')
plt.title('Grayscale Frame')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(edges, cmap='gray')
plt.title('Canny Edges')
plt.axis('off')

plt.show()
```



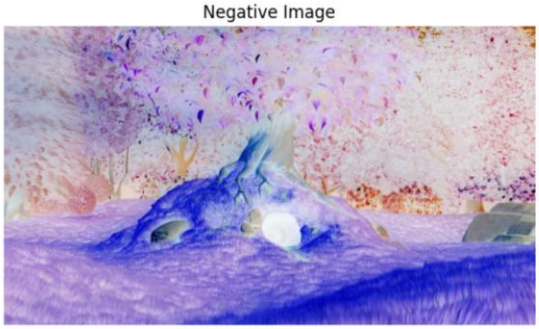
Creating a Negative Image from RGB

We will create a negative of the `frame_to_draw_rgb` (the first frame we processed) by inverting its pixel intensities.

```
# To create a negative image, we invert the pixel values.
# For an 8-bit image (0-255), this means 255 - pixel_value.
# We'll use the 'frame_to_draw_rgb' which was the original frame (in RGB format).
negative_image = 255 - frame_to_draw_rgb

# Display the original RGB frame and its negative
plt.figure(figsize=(15, 7))

plt.subplot(1, 2, 1)
plt.imshow(frame_to_draw_rgb)
plt.title('Original RGB Frame')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(negative_image)
plt.title('Negative Image')
plt.axis('off')

plt.show()
```

Original RGB Frame

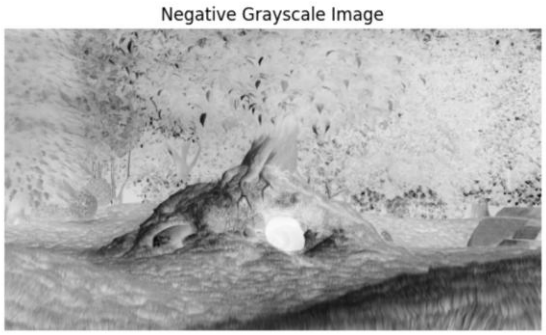
Negative Image



Creating a Negative Image from Grayscale

As requested, we will now create a negative of the `frame_gray` (the grayscale version of our first frame) by inverting its pixel intensities. This will show how brightness values are inverted in a single channel image.

```
# Ensure frame_gray exists from previous steps, or re-create it if not.
# For an 8-bit grayscale image (0-255), this means 255 - pixel_value.

# We will use the 'frame_gray' that was created after converting frame_to_draw to grayscale.
# If frame_gray is not defined, you might need to re-run the 'Convert to Grayscale' step.
if 'frame_gray' in locals():
    negative_grayscale_image = 255 - frame_gray

    # Display the original grayscale frame and its negative
    plt.figure(figsize=(15, 7))

    plt.subplot(1, 2, 1)
    plt.imshow(frame_gray, cmap='gray')
    plt.title('Original Grayscale Frame')
    plt.axis('off')

    plt.subplot(1, 2, 2)
    plt.imshow(negative_grayscale_image, cmap='gray')
    plt.title('Negative Grayscale Image')
    plt.axis('off')

    plt.show()
else:
    print("Error: 'frame_gray' is not defined. Please ensure the grayscale conversion step was executed.")
```




```
cap = cv2.VideoCapture('/content/sample_video.mp4')

if cap.isOpened():
    print("Video capture object successfully re-initialized.")
else:
    print("Error: Could not re-initialize video capture object.")
```

```
Video capture object successfully re-initialized.
```

Reasoning: Before processing and writing frames to a new video file, it's essential to retrieve the original video's properties (width, height, frames per second) to correctly configure the `cv2.VideoWriter` object for the output video. These properties will ensure the output video has the same dimensions and frame rate as the input.

```
fps = cap.get(cv2.CAP_PROP_FPS)
width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))

print(f"Original video properties: FPS={fps}, Width={width}, Height={height}")
```

```
Original video properties: FPS=60.0, Width=1920, Height=1080
```

Reasoning: To prepare for writing the processed frames to a new video file, it's necessary to set up a `cv2.VideoWriter` object. This involves specifying the output filename, the video codec, the frame rate, and the frame dimensions. Since the output will be a grayscale video, the `isColor` flag will be set to `False`.


```
output_video_path = '/content/negative_grayscale_video.mp4'
fourcc = cv2.VideoWriter_fourcc(*'mp4v') # Changed codec to 'mp4v' for better browser compatibility
out = cv2.VideoWriter(output_video_path, fourcc, fps, (width, height), isColor=False)

if out.isOpened():
    print(f"VideoWriter successfully opened for {output_video_path}")
else:
    print(f"Error: Could not open VideoWriter for {output_video_path}")
```

```
VideoWriter successfully opened for /content/negative_grayscale_video.mp4
```

Reasoning: Now that the video capture object and video writer are initialized, the next step is to iterate through each frame of the input video. For each frame, it needs to be converted to grayscale, then its pixel intensities inverted to create a negative grayscale image, and finally, this processed frame written to the output video file.

```
frame_count = 0
success_count = 0

while True:
    ret, frame = cap.read()
    if not ret:
        break # Break the loop if no more frames to read

    # Convert frame to grayscale
    gray_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    # Create negative grayscale image
    negative_gray_frame = 255 - gray_frame

    # Write the processed frame to the output video
    out.write(negative_gray_frame)
```

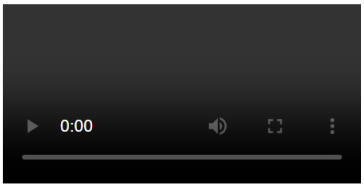
```
        frame_count += 1
        success_count += 1

        if frame_count % 100 == 0:
            print(f"Processed {frame_count} frames.")

    print(f"Finished processing. Total frames processed: {success_count}")

    # Release the video capture and writer objects
    cap.release()
    out.release()

    print("Video capture and writer objects released.")
```

```
Processed 100 frames.
Processed 200 frames.
Processed 300 frames.
Processed 400 frames.
Processed 500 frames.
Processed 600 frames.
Finished processing. Total frames processed: 600
Video capture and writer objects released.
```

Displaying the Negative Grayscale Video

Here, we will use `IPython.display.Video` to embed and play the generated `/content/negative_grayscale_video.mp4` directly within the notebook.

```
from IPython.display import Video, display

video_path = '/content/negative_grayscale_video.mp4'
display(Video(video_path, embed=True, html_attributes='controls autoplay'))
```



Backend of Number Plate Detection System

Tech Stack:

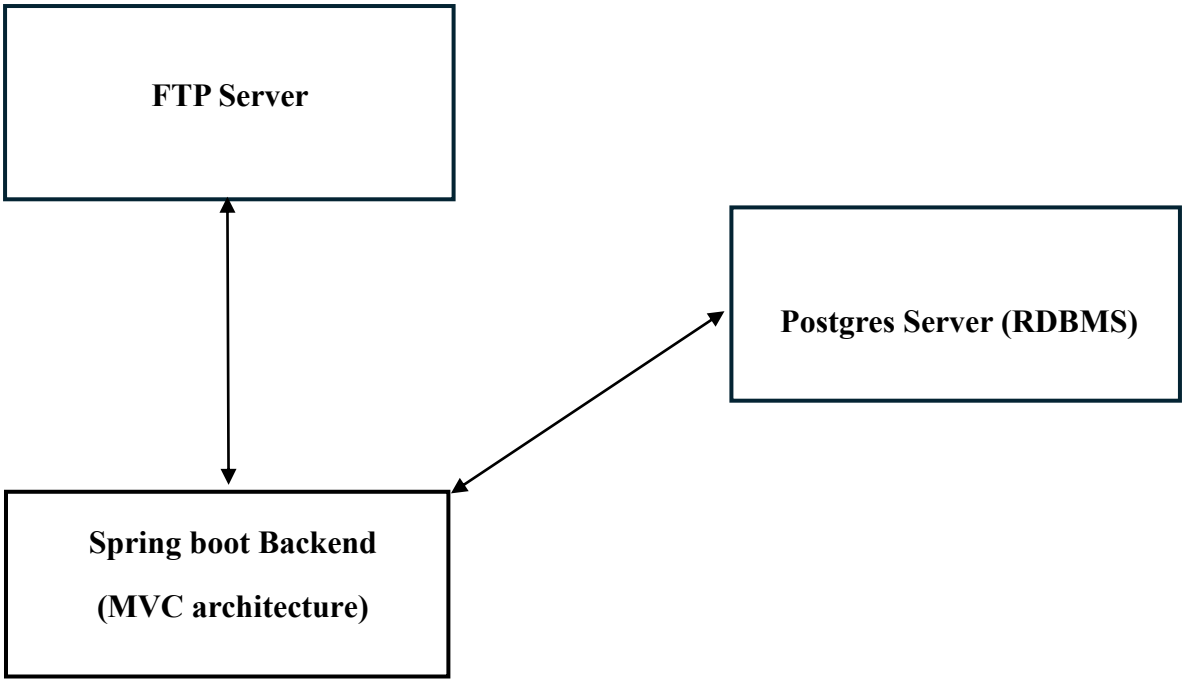
- Spring Boot (Microservices Architecture)
- MVC architecture
- Hibernate / PVA
- PostgreSQL
- Postman for API testing

Main Components:

- Model layer

- Repository layer
- Service layer
- Controller layer
- Database layer
- API testing layer

Architecture diagram:



Complete Workflow:

1. **Camera** captures vehicle image
2. Number plate detection **model** processes the image
3. Detected number plate + image path + timestamps are generated
4. Backend **Spring Boot API** receives this data
5. **Controller** receives Request
6. Controller calls **Service Layer**
7. Service processes the data
8. Repository stores the data using **Hibernate / JPA**
9. Hibernate automatically generates **SQL queries**
10. Data stored in **PostegreSQL** Database
11. **APIs** are tested using Postman.

Key Advantages:

i. Automatic table creation using Hibernate
ii. No need to write SQL queries manually
iii. Clean Architecture using MVC
iv. Easy API testing using Postman
v. Scalable microservice backend

Challenges faced and Resolutions:

During development, a significant challenge arose concerning FTP directory traversal. Standard navigation logic caused the client to lose its relative position when attempting to upload files into newly created, deeply nested directories. This was successfully resolved by implementing a rigid state-management protocol within the service layer, ensuring the working directory is reset to the server root before constructing and entering the required path structure. Furthermore, system configurations were heavily tailored to resolve Windows IIS FTP authentication hurdles, explicitly mapping local Windows user permissions to the Java client's read/write requests.